

DJANGO REST FRAMEWORK

JSON Web Token amb DRF

Teoria, conceptes i implementació pas a pas

Login segur · Serializers · @api_view · Django MVT

Django 5.0

DRF

PyJWT

Python 3.10+

Índex

1. Introducció al JWT

- 1.1 Què és un JWT?
- 1.2 Estructura del token
- 1.3 Flux de funcionament
- 1.4 JWT vs Sessions de Django

2. JWT en Django REST Framework

- 2.1 Rol de DRF en l'autenticació JWT
- 2.2 Serializers per al login
- 2.3 @api_view i la resposta JSON
- 2.4 Guardar el token a la base de dades
- 2.5 Integració MVT + DRF
- 2.6 La funció authenticate() de Django

3. Pas a pas: Implementació completa

- Pas 1 – Instal·lació i estructura
- Pas 2 – Configuració (settings.py)
- Pas 3 – Model TokenJWT (models.py)
- Pas 4 – Serializer (serializers.py)
- Pas 5 – Views (views.py)
- Pas 6 – URLs (urls.py)
- Pas 7 – Templates HTML
- Pas 8 – Migrar i executar

4. Resum i flux complet

1. Introducció al JWT

1.1 Què és un JWT?

Un **JWT (JSON Web Token)** és un estàndard obert (*RFC 7519*) per transmetre informació de manera segura entre dues parts com un objecte JSON signat digitalment. S'utilitza principalment per a **l'autenticació i l'autorització** en aplicacions web i APIs.

La idea central és senzilla: quan un usuari s'autentica correctament, el servidor li lliura un **token**. A partir d'aquell moment, l'usuari presenta aquest token en cada petició per demostrar la seva identitat, *sense que el servidor hagi de consultar la base de dades cada vegada*.

Analogia: El JWT funciona com un carnet d'identitat digital. El servidor l'emet en el moment del login. L'usuari el guarda i el presenta quan ho necessita. El servidor només ha de verificar que és autèntic (signatura vàlida) i que no ha caducat.

1.2 Estructura del token

Un JWT és un text format per **tres parts separades per punts**, cadascuna codificada en Base64:

eyJhbGciOiJIUzI1NiJ9

.eyJ1c2VybmFtZSI6InJvZ2VyIn0

.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c

Part	Nom	Contingut (decodificat)
Vermell	Header	{"alg": "HS256", "typ": "JWT"} — algorisme de signatura
Morat	Payload	{"username": "roger", "exp": 1234567890} — dades de l'usuari
Verd	Signature	HMAC-SHA256 del header + payload, signat amb la SECRET_KEY

Important: El Payload és codificat en Base64 però **NO xifrat**. Qualsevol pot decodificar-lo i llegir el contingut. La Signature garanteix que el token *no ha estat modificat*, però no amaga la informació. Per tant, **mai s'hi ha de guardar la contrasenya ni dades sensibles**.

Claims del Payload

El Payload conté *claims* (afirmacions). N'hi ha d'estàndard i de personalitzats:

Claim	Significat	Exemple
sub	Subject (identificador de l'usuari)	"sub": "42"
exp	Expiration (data de caducitat, Unix timestamp)	"exp": 1700000000
iat	Issued At (quan s'ha creat el token)	"iat": 1699996400
username	Claim personalitzat (nom d'usuari)	"username": "roger"

1.3 Flux de funcionament

- 1 Login:** L'usuari envia usuari + contrasenya al servidor (POST).
- 2 Verificació:** El servidor comprova les credencials contra la base de dades (`authenticate()`).
- 3 Generació:** Si les credencials són correctes, el servidor crea el JWT amb `jwt.encode(payload, SECRET_KEY)` i el signa.
- 4 Emmagatzematge:** El token es guarda a la BD (model `TokenJWT`) i s'envia al client (sessió/cookie).
- 5 Ús:** En cada petició, el client presenta el token. El servidor el verifica amb `jwt.decode()` i extreu les dades del Payload.
- 6 Caducitat:** Si el token ha caducat (`ExpiredSignatureError`), el servidor rebutja la petició i l'usuari ha de tornar a fer login.

1.4 JWT vs Sessions de Django

Característica	Sessions Django	JWT
On s'emmagatzema	Servidor (BD o memòria)	Client (cookie / localStorage)
Consulta a la BD per validar	Sí, a cada petició	No (el token és autocontingut)
Escalabilitat (múltiples servidors)	Requereix sessió compartida	No requereix estat al servidor
Ús en APIs REST	No és l'estàndard	Estàndard de facto
Revocar accés	Senzill (esborrar sessió)	Cal guardar tokens a la BD
Informació inclosa	Només l'ID de sessió	Dades de l'usuari al Payload

2. JWT en Django REST Framework

2.1 Rol de DRF en l'autenticació JWT

Django REST Framework (DRF) és un conjunt d'eines per construir APIs web amb Django. Quan s'implementa JWT, DRF fa dos papers fonamentals:

- **Validar les dades d'entrada** (usuari i contrasenya) gràcies als *Serializers*.
- **Exposar l'endpoint de login** com una ruta API que retorna JSON, gràcies al decorador `@api_view`.

La lògica de generar el token JWT (amb la llibreria `PyJWT`) i de guardar-lo a la base de dades és responsabilitat del codi Python que escrivim a `views.py`, no de DRF en si. DRF proporciona l'estructura per exposar-ho com una API.

Combinació MVT + DRF: En aquest projecte fem servir *dos patrons alhora*. DRF gestiona l'endpoint de login com a API (retorna JSON). Django MVT gestiona les pàgines HTML (login form i pàgina de benvinguda). Tots dos comparteixen la mateixa lògica de generació del token.

2.2 Serializers per al login

Seguint la teoria de DRF, el **Serializer** és l'equivalent del `ModelForm` de Django però orientat a APIs. S'encarrega de:

- Rebre les dades de la petició (`request.data`).
- Validar que els camps requerits hi siguin i tinguin el format correcte.
- Retornar errors si les dades no són vàlides (`serializer.errors`).

Per al login, el serializer és molt senzill: només necessita `username` i `password` :

`auth_app/serializers.py`

```
from rest_framework import serializers

class LoginSerializer(serializers.Serializer):
    username = serializers.CharField(required=True)
    password = serializers.CharField(required=True)
```

Per què no ModelSerializer? El `ModelSerializer` s'usa quan les dades corresponen directament a un model de la BD. Aquí, `password` no s'emmagatzema directament, sinó que s'usa per autenticar. Per tant, s'usa el `Serializer` base i es declaren els camps manualment.

2.3 @api_view i la resposta JSON

`@api_view` és el decorador de DRF que transforma una funció Python normal en un endpoint API. Indica quins mètodes HTTP accepta i formata la resposta com a JSON:

```
@api_view(['POST'])
def login_api(request):
    serializer = LoginSerializer(data=request.data) # rep les dades
    if serializer.is_valid():                       # valida
        ...
        return Response({'token': token})          # retorna JSON
    return Response(serializer.errors, status=400) # errors de validació
```

El flux dins la vista és:

- 1 Crear el serializer amb `LoginSerializer(data=request.data)`.
- 2 Cridar `serializer.is_valid()` per validar els camps.
- 3 Extreure les dades validades amb `serializer.validated_data`.
- 4 Autenticar amb `authenticate(username, password)` (funció de Django).
- 5 Generar el JWT i retornar-lo amb `Response({'token': token})`.

2.4 Guardar el token a la base de dades

Per poder **revocar tokens** o auditar els accessos, és una bona pràctica guardar cada token generat a la base de dades. Es crea un model `TokenJWT` que relaciona un token amb l'usuari que l'ha generat.

Seguint la teoria de Django Models, s'usa una **ForeignKey** per establir la relació *molts a un* entre tokens i usuari (un usuari pot tenir molts tokens, però cada token pertany a un únic usuari):

```
auth_app/models.py
```

```

from django.db import models
from django.contrib.auth.models import User

class TokenJWT(models.Model):
    user      = models.ForeignKey(User, on_delete=models.CASCADE)
    token     = models.TextField()
    creat_el  = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"Token de {self.user.username}"

```

Camp	Tipus	Descripció
user	ForeignKey → User	Usuari propietari del token. CASCADE: si s'elimina l'usuari, el token també.
token	TextField	El token JWT complet (cadena llarga, per això TextField).
creat_el	DateTimeField	Data de creació automàtica (auto_now_add).

2.5 Integració MVT + DRF

El projecte combina dos enfocaments en el mateix `views.py`:

Patró	Vista	Retorna	Usat per
DRF	login_api	JSON {'token': '...', 'username': '...'}	Postman, apps mòbils, JavaScript
MVT	login_page	Redirect → welcome.html	Formulari HTML del navegador
MVT	welcome_page	Template HTML amb {{ username }}	Pàgina de benvinguda

La vista `welcome_page` extreu el nom d'usuari **directament del payload JWT** desat a la sessió, sense consultar la base de dades:

```

token    = request.session.get('jwt_token')      # llegim de la sessió
payload  = jwt.decode(token, settings.SECRET_KEY, ...) # decodifiquem el JWT
username = payload['username']                   # extraiem el nom

```

2.6 La funció authenticate() de Django

`authenticate(username=..., password=...)` és una funció de `django.contrib.auth` que verifica les credencials d'un usuari contra la base de dades. Retorna l'objecte `User` si les credencials són correctes, o `None` en cas contrari.

```
user = authenticate(username="roger", password="1234")
if user:
    # credencials correctes → user és l'objecte User de Django
else:
    # credencials incorrectes → user és None
```

Per què és important? Mai s'han de comparar contrasenyes manualment. Django les emmagatzema **hashejades** (PBKDF2/bcrypt) i `authenticate()` s'encarrega de tot el procés: buscar l'usuari a la BD, hashear la contrasenya rebuda i comparar. Si ho féssim manualment arriscaríem errors greus de seguretat.

Situació	Retorn
Usuari existeix i contrasenya correcta	Objecte User
Usuari no existeix	None
Contrasenya incorrecta	None
Usuari inactiu (<code>is_active=False</code>)	None

3. Pas a Pas: Implementació completa

Pas 1 — Instal·lació i estructura del projecte

Instal·lació de les llibreries

```
pip install django
pip install djangorestframework
pip install PyJWT
```

Crear el projecte i l'app

```
django-admin startproject projecte .
python3 manage.py startapp auth_app
```

Estructura de fitxers

```
projecte_jwt/
├── projecte/
│   ├── settings.py    ← configuració global
│   └── urls.py        ← URLs principals
└── auth_app/
    ├── models.py      ← model TokenJWT
    ├── serializers.py  ← LoginSerializer (DRF)
    ├── views.py       ← vistes API + MVT
    ├── urls.py        ← URLs de l'app
    └── templates/
        ├── base.html
        ├── login.html
        └── welcome.html
```

Pas 2 — Configuració (settings.py)

Afegir `rest_framework` i `auth_app` a `INSTALLED_APPS`, i configurar els templates:

```
projecte/settings.py (fragments rellevants)
```

```

INSTALLED_APPS = [
    # apps Django per defecte...
    'rest_framework',    # DRF
    'auth_app',          # la nostra aplicació
]

TEMPLATES = [{
    'BACKEND': 'django.template.backends.django.DjangoTemplates',
    'DIRS': [BASE_DIR / 'auth_app/templates'],
    'APP_DIRS': True,
    ...
}]

```

Pas 3 — Model TokenJWT (models.py)

El model guarda cada token generat associat a l'usuari. Segueix el patró de Django Models amb una **ForeignKey** cap al model `User` integrat de Django:

auth_app/models.py

```

from django.db import models
from django.contrib.auth.models import User

class TokenJWT(models.Model):
    user      = models.ForeignKey(User, on_delete=models.CASCADE)
    token     = models.TextField()
    creat_el  = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"Token de {self.user.username}"

```

`on_delete=models.CASCADE` : si s'elimina l'usuari, tots els seus tokens també s'eliminen.
`auto_now_add=True` : la data de creació es guarda automàticament en crear el registre.

Pas 4 — Serializer (serializers.py)

El serializer valida que la petició de login contingui `username` i `password`. Segueix el patró DRF: hereta de `serializers.Serializer` i declara els camps amb els seus tipus i restriccions:

auth_app/serializers.py

```
from rest_framework import serializers

class LoginSerializer(serializers.Serializer):
    username = serializers.CharField(required=True)
    password = serializers.CharField(required=True)
```

Pas 5 — Views (views.py)

El fitxer `views.py` conté quatre vistes. La primera segueix el patró DRF (amb `@api_view`); les altres tres segueixen el patró Django MVT (amb `render` i `redirect`):

```
auth_app/views.py
```

```

import jwt
import datetime

from django.conf import settings
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate
from rest_framework.decorators import api_view
from rest_framework.response import Response

from .models import TokenJWT
from .serializers import LoginSerializer

# — Vista API (DRF) —————

@api_view(['POST'])
def login_api(request):
    serializer = LoginSerializer(data=request.data)
    if serializer.is_valid():
        username = serializer.validated_data['username']
        password = serializer.validated_data['password']
        user = authenticate(username=username, password=password)
        if user:
            token = _generar_token(user)
            TokenJWT.objects.create(user=user, token=token)
            return Response({'token': token, 'username': user.username})
        return Response({'error': 'Credenciales incorrectes'}, status=400)
    return Response(serializer.errors, status=400)

# — Vistes MVT (Django) —————

def login_page(request):
    error = None
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')
        user = authenticate(request, username=username, password=password)
        if user:
            token = _generar_token(user)
            TokenJWT.objects.create(user=user, token=token)
            request.session['jwt_token'] = token
            return redirect('welcome')
        error = 'Usuari o contrasenya incorrectes'
    return render(request, 'login.html', {'error': error})

def welcome_page(request):
    token = request.session.get('jwt_token')
    if not token:
        return redirect('login')
    try:
        payload = jwt.decode(token, settings.SECRET_KEY, algorithms=['HS256'])

```

```

        username = payload['username']
    except jwt.ExpiredSignatureError:
        return redirect('login')
    except jwt.InvalidTokenError:
        return redirect('login')
    return render(request, 'welcome.html', {'username': username})

def logout_page(request):
    token = request.session.get('jwt_token')
    if token:
        TokenJWT.objects.filter(token=token).delete() # invalida el token a la BD
        del request.session['jwt_token'] # esborra de la sessió
    return redirect('login')

# — Funció auxiliar —

def _generar_token(user):
    payload = {
        'username': user.username,
        'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)
    }
    return jwt.encode(payload, settings.SECRET_KEY, algorithm='HS256')

```

Què fa login_page pas a pas?

- 1 **GET inicial:** L'usuari accedeix a `/login/`. Com que no és POST, s'executa directament el `render` del final i es mostra el formulari buit. `error = None` fa que no aparegui cap missatge d'error.
- 2 **POST amb el formulari:** L'usuari escriu les credencials i prem "Entrar". El navegador envia una petició POST. S'entra al bloc `if request.method == 'POST'` i s'extreuen `username` i `password` de `request.POST`.
- 3 **Verificació de credencials:** `authenticate()` busca l'usuari a la BD i compara la contrasenya (hashejada). Si les credencials són incorrectes retorna `None`, s'assigna el missatge d'error i es torna a renderitzar el formulari amb l'error visible.
- 4 **Credencials correctes:** Es crida `_generar_token(user)` per crear el JWT amb el nom d'usuari i una expiració d'1 hora. El token es guarda a la BD (`TokenJWT.objects.create`) per poder-lo revocar si cal.
- 5 **Desar a la sessió i redirigir:** El token es guarda a `request.session['jwt_token']` — Django l'emmagatzema al servidor associat a una cookie de sessió al navegador. Finalment, `redirect('welcome')` envia l'usuari a la pàgina de benvinguda.

Què fa welcome_page pas a pas?

- 1 **Recuperar el token:** `request.session.get('jwt_token')` intenta llegir el token desat a la sessió. Si no existeix (l'usuari no ha fet login o la sessió ha expirat), es redirigeix directament a login.
- 2 **Decodificar el JWT:** `jwt.decode(token, SECRET_KEY, algorithms=['HS256'])` verifica la signatura del token i n'extreu el payload. Si la signatura no coincideix (token manipulats), llança `InvalidTokenError`.
- 3 **Gestió d'errors del token:** Si el token ha caducat (`ExpiredSignatureError`) o és invàlid per qualsevol altra raó (`InvalidTokenError`), es redirigeix a login. Aquesta és la protecció contra tokens falsificats o caducats.
- 4 **Mostrar la pàgina:** Si el token és vàlid, s'extreu el `username` del payload i es passa al template `welcome.html` via el diccionari de context. El nom d'usuari s'ha obtingut **sense fer cap consulta a la BD**, només decodificant el JWT.

`_generar_token(user)`: funció auxiliar que construeix el payload amb el nom d'usuari i una expiració d'1 hora. La `SECRET_KEY` de Django s'usa per signar el token. Qualsevol modificació del token serà detectada en fer el `decode`.

`logout_page(request)`: elimina el token de la base de dades (invalida l'accés fins i tot si algú el tingués copiat) i l'esborra de la sessió. Finalment redirigeix a la pàgina de login. Aquesta doble eliminació és la manera correcta de tancar sessió amb JWT.

Pas 6 — URLs (urls.py)

Seguint el patró Django MVT, l'arxiu `urls.py` de l'app mapeja cada path al seu mètode de `views`:

`auth_app/urls.py`

```
from django.urls import path
from . import views

urlpatterns = [
    path('login/', views.login_page, name='login'),
    path('welcome/', views.welcome_page, name='welcome'),
    path('logout/', views.logout_page, name='logout'),
    path('api/login/', views.login_api, name='login_api'),
]
```

L'arxiu `urls.py` del **projecte** inclou les URLs de l'app amb `include`:

`projecte/urls.py`

```

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('auth_app.urls')),
]

```

Pas 7 — Templates HTML

Les plantilles segueixen el patró Django Templates: una `base.html` com a plantilla base, i les altres hereten d'ella amb `{% extends %}`.

`auth_app/templates/base.html`

```

<!DOCTYPE html>
<html lang="ca">
<head>
    <meta charset="UTF-8">
    <title>JWT Login</title>
</head>
<body>
    {% block content %}
    {% endblock %}
</body>
</html>

```

`auth_app/templates/login.html`

```

{% extends 'base.html' %}

{% block content %}
<h1>Iniciar Sessió</h1>

{% if error %}
    <p style="color:red;">{{ error }}</p>
{% endif %}

<form action="" method="POST">
    {% csrf_token %}
    <label>Usuari: <input type="text" name="username"></label><br>
    <label>Contrasenya: <input type="password" name="password"></label><br>
    <input type="submit" value="Entrar">
</form>
{% endblock %}

```

`auth_app/templates/welcome.html`


```
{% extends 'base.html' %}

{% block content %}
<h1>Benvingut, {{ username }}!</h1>
<p>El teu nom s'ha extret del token JWT desat a la sessió.</p>
<a href="{% url 'logout' %}">Tancar sessió</a>
{% endblock %}
```

`{{ username }}` : variable passada des de `welcome_page` al template. El seu valor prové de `payload['username']` extret del JWT, **no d'una consulta a la BD**.

Pas 8 — Migrar i executar

```
# Crear les migracions del model TokenJWT
python3 manage.py makemigrations

# Aplicar les migracions a la base de dades
python3 manage.py migrate

# Crear un superusuari per fer proves
python3 manage.py createsuperuser

# Iniciar el servidor
python3 manage.py runserver
```

Accedeix a <http://127.0.0.1:8000/login/> per veure el formulari. Introdueix les credencials del superusuari creat. Si les credencials són correctes, seràs redirigit a `/welcome/` on apareixerà el teu nom extret del token JWT.

4. Resum i flux complet

Flux de la vista MVT (formulari HTML)

#	Fitxer	Acció
1	auth_app/urls.py	La URL login/ crida la vista login_page
2	views.py → login_page	Rep el POST del formulari (username + password)
3	views.py → login_page	Crida authenticate() per verificar les credencials
4	views.py → _generar_token	Genera el JWT amb PyJWT (payload + SECRET_KEY)
5	models.py → TokenJWT	Guarda el token a la base de dades (objects.create)
6	views.py → login_page	Desa el token a request.session['jwt_token']
7	views.py → login_page	Fa redirect('welcome')
8	views.py → welcome_page	Llegeix el token de la sessió, fa jwt.decode()
9	templates/welcome.html	Mostra {{ username }} i enllaç "Tancar sessió"
10	auth_app/urls.py	La URL logout/ crida la vista logout_page
11	views.py → logout_page	Elimina el token de la BD i de la sessió, redirigeix a login

Flux de la vista API (DRF, per a Postman o JavaScript)

#	Fitxer	Acció
1	auth_app/urls.py	La URL api/login/ crida la vista login_api
2	views.py → login_api	Rep request.data (JSON amb username + password)
3	serializers.py	LoginSerializer.is_valid() valida els camps
4	views.py → login_api	Autentica, genera token, guarda a TokenJWT
5	views.py → login_api	Retorna Response({'token': ..., 'username': ...})

Resum dels conceptes clau

Concepte	Fitxer	Rol
JWT	—	Token autocontingut que identifica l'usuari sense consultar la BD
TokenJWT (Model)	models.py	Registre del token a la BD per auditar i revocar
LoginSerializer	serializers.py	Valida les dades d'entrada (username + password)
@api_view	views.py	Transforma la funció en un endpoint API que retorna JSON
login_page	views.py	Vista MVT: rep formulari, genera JWT, redirigeix
welcome_page	views.py	Vista MVT: llegeix JWT de la sessió, mostra username
logout_page	views.py	Vista MVT: elimina el token de la BD i sessió, redirigeix a login
login.html	templates/	Formulari de login amb {% csrf_token %}
welcome.html	templates/	Mostra {{ username }} i enllaç de logout